

简单字符串算法

Part I

wYYSZL

2026.8.12

Outline

主要讲基础的字符串算法，包括字符串哈希、KMP、Manacher、AC 自动机等。
假定大家对于入门组的内容已经比较熟练了，所以这部分内容会讲的快一点。
带星的题目是选做题目，但是多数也不会特别难。
欢迎大家挑战它们。

本节课内容

- ① 字符串哈希
- ② KMP 算法
- ③ Border 的性质与单串失配树

记号约定

本讲义全文采用以下记号约定：

1. **下标从 1 开始**： s_1 为字符串的第一个字符， s_i 表示第 i 个字符；当下标不在范围内时，对应位置的字符视为空字符。
2. **长度记号**： $|s|$ 、 $|S|$ 、 $|T|$ 分别表示对应字符串 s 、 S 、 T 的长度； len 也常用作长度（ $len = |s|$ ）。
3. **子串**：由 s 在开头或末尾删去若干字符得到的字符串称为 s 的子串， s 本身和空串均为 s 的子串； $s[l..r]$ 或 $s_{l\sim r}$ 表示 s 位置 l 到 r 上的字符连接而成的子串，当 $l > r$ 时为空串。

目录

① 字符串哈希

② KMP 算法

③ Border 的性质与单串失配树

哈希

我们总会遇到一些难以判断是否相同的东西，例如很长的字符串、有一堆变量的结构体。这些东西需要逐个去看是否相同，非常慢。我们希望有一个快一些的做法。一种常见的思想叫做哈希：

哈希 (Hash)

哈希的核心思想在于，将输入映射到一个值域较小、方便比较的范围。

对于字符串哈希，就是用一个函数，将一个字符串变成一个整数，这个函数，或者说对应方法就是**哈希函数**，对应得到的整数就是**哈希值**。

哈希函数

哈希函数通常要满足这两个条件：

- 在 Hash 函数值不一样的时候，两个字符串一定不一样；
- 在 Hash 函数值一样的时候，两个字符串大概率一样（且我们当然希望它们总是一样的）。

第一个是函数带来的自然性质，第二个是关键，我们之后会讨论。

字符串哈希

字符串哈希有相对固定的哈希函数。

我们将一个字符串 s 看成一个 B 进制数，我们这里认为 s_1 为最高位，则它对应的值就是 $s_1 B^{len-1} + s_2 B^{len-2} + \dots s_{len} B^0$ 。但这个数字太大，比较这个数字基本相当于又完整比较了一遍字符串。所以我们将这个数对 MOd 取模，得到最终的哈希值。也就是说，哈希函数定义为：

$$f(s) = \sum_{i=1}^{len} s_i \times B^{len-i} \bmod MOd$$

B, MOd 的选择是任意的，但通常也有一定偏好。

字符串哈希

回看我们刚才的要求：

- 在 Hash 函数值不一样的时候，两个字符串一定不一样；
- 在 Hash 函数值一样的时候，两个字符串大概率一样（希望总是一样的）。

第一条显然满足，一个函数不可能有两个函数值。但第二条就不一定了。如果出现哈希值相同但是字符串不同，也就是说两个不同的字符串共用一个哈希值，我们称之为**哈希冲突**。

我们希望避免哈希冲突。

字符串哈希 - 哈希冲突

我们希望避免哈希冲突。

最显然地，我们希望不取模是不会冲突的，所以 B 最好大于 $\max s_i$ 。此外，我们值域大一点，取 MOd 为质数，或至少 MOd 和 B 互质会好。

实践中，我们通常取 $10^9 + 7, 998244353$ 这样的大质数为 MOd ，且它们不会过大不方便计算。有时也取 2^{64} 这样的 MOd ，虽然不是质数但很大，可以自然溢出。

为了互质，我们偷懒地直接令 B 为质数，如 131, 13331 等。

字符串哈希 - 哈希冲突

但这样不够，由于生日悖论，我们知道当有 10^6 ($\ll 10^9 + 7$) 个字符串的时候，就有极高的概率会出现哈希冲突。

此外，也有具体的方法可以构造自然溢出式哈希的哈希冲突。

为了处理这些问题，我们用**多哈希**的技巧。具体地，我们算两个或多个哈希值，每个哈希值取的 Mod 不同，只有多个哈希值都相同时才认为字符串相同。这样可以极大降低冲突的可能。

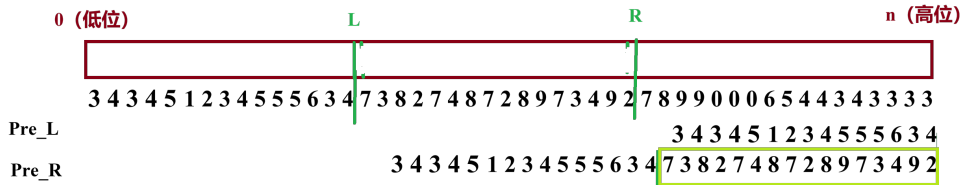
另外，为了保证不被特殊构造，可以用一些非常见的 B 和 Mod ，但要保证上面的限制。

一种理解

非构造性的哈希冲突的原因可以简单理解为 Mod 不够大，而取双/多哈希实际上将 Mod 扩展为了所有 Mod 的 LCM。

字符串哈希 - 应用

字符串哈希，或者说采用将字符串看成 B 进制数的“进制位哈希”，更多应用于给定一个长字符串，快速得到下标在 $[l, r]$ 的子串的哈希，方便与其他东西比较。不要忘记现在字符串是一个 B 进制数，这样我们就能想到解法。



设 pre_i 表示前 i 个字符构成的哈希，即 $pre_i = (s_i + s_{i-1}B + \dots) \bmod Mod$ 。则 pre_r 是 B 进制下的 r 位数，其低 $r - l + 1$ 位即 $[l, r]$ 。于是 $[l, r]$ 的哈希值为 $pre_r - pre_{l-1}B^{r-l+1}$ 。可以预处理 B^i 以便实现 $\mathcal{O}(1)$ 求解。

字符串哈希

```
1 pow1[0] = pow2[0] = 1;
2 for (int i = 1; i <= n; ++i) {
3     pow1[i] = pow1[i - 1] * BASE % MOD1;
4     pow2[i] = pow2[i - 1] * BASE % MOD2;
5
6     // 这里将字符映射为 1~26, 也可用 ASCII 码
7     int val = s[i - 1] - 'a' + 1;
8     pre1[i] = (pre1[i - 1] * BASE + val) % MOD1;
9     pre2[i] = (pre2[i - 1] * BASE + val) % MOD2;
10 }
11
12
13 // 查询子串 s[l..r] 的哈希值, 返回 pair<hash1, hash2>, 下标从 0 开始
14 pair<long long, long long> get_hash(int l, int r) {
15     long long h1 = (pre1[r + 1] - pre1[l] * pow1[r - l + 1] % MOD1 + MOD1) % MOD1;
16     long long h2 = (pre2[r + 1] - pre2[l] * pow2[r - l + 1] % MOD2 + MOD2) % MOD2;
17     return {h1, h2};
18 }
```

图: 实现示例

例题 A

A - 【模板】字符串哈希

给定 N 个字符串，请求出 N 个字符串中共有多少个不同的字符串。

例题 A

A - 【模板】字符串哈希

给定 N 个字符串，请求出 N 个字符串中共有多少个不同的字符串。

模板题。

例题 B

B - 兔子与兔子

给定字符串 S ，长度为 n 。 m 次询问，每次给出四个下标 l_1, r_1, l_2, r_2 ，判断子串 $S[l_1..r_1]$ 与 $S[l_2..r_2]$ 是否完全相等。

数据规模： $n, m \leq 10^6$ 。

例题 B

B - 兔子与兔子

给定字符串 S ，长度为 n 。 m 次询问，每次给出四个下标 l_1, r_1, l_2, r_2 ，判断子串 $S[l_1..r_1]$ 与 $S[l_2..r_2]$ 是否完全相等。

数据规模： $n, m \leq 10^6$ 。

模板题 2.0。

例题 C

C - Compress Words

Amugae 有一个由 n 个单词组成的句子。他想把这个句子压缩成一个单词。Amugae 不喜欢重复，因此当他将两个单词合并成一个单词时，会移除第二个单词中与第一个单词后缀相同的最长前缀。例如，他将"sample" 和"please" 合并成"samplease"。Amugae 会从左到右依次合并他的句子（即，先合并前两个单词，然后将结果与第三个单词合并，依此类推）。请编写程序输出合并过程结束后得到的压缩单词。

数据规模： $n \leq 10^5$ ， $\sum |s_i| \leq 10^6$ 。

例题 C 题解

我们用哈希来判断一个前缀和后缀是否相同。

现在要加入一个新的串 T ，从大到小枚举前后缀长度，然后判断是否相同。这样做的复杂度为 $\mathcal{O}(len)$ ，所以直接这样做即可。

注意合并后，字符串会改变，哈希也会跟着变。一个最简单的办法是，在合并的时候再算哈希，并且只需要算新串 T 长度的哈希即可。

目录

① 字符串哈希

② KMP 算法

③ Border 的性质与单串失配树

KMP 算法要解决的是这样的问题：

字符串匹配

假设有两个字符串 S 和 T ，寻找 S 在 T 的哪些位置出现。

KMP 算法要解决的是这样的问题：

字符串匹配

假设有两个字符串 S 和 T ，寻找 S 在 T 的哪些位置出现。

首先最暴力的做法就是枚举 T 中的每一个位置，然后遍历看从这之后的 len_S 个字符是否和 S 一样。设 T, S 的长度为 n, m ，这个算法的复杂度是 $\mathcal{O}(nm)$ 。

显然，我们在比较的时候，可以使用哈希而不是暴力比较，这样可以实现 $\mathcal{O}(n + m)$ 的复杂度。但我们现在要讲另一种做法：

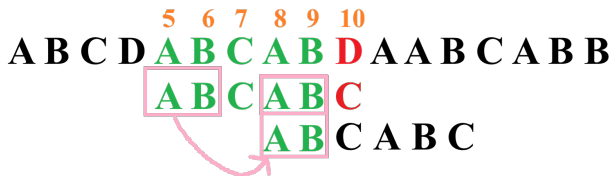
在刚才的暴力中，当看到某个字符与 S 对应位置的字符不一样时，就会结束比较，然后到下一个位置从 S_1 开始继续比较。前面的比较结果不能对后面的比较起到指导作用，显然是不优秀的。

由于已经匹配了一部分，所以我们对 T 是有一定把握的，或者说，我们知道 T 与 S 的一定相似性。因而，如果我们能对 S 进行处理，说不定就能避免一些无用的比较。

5 6 7 8 9 10
A B C D A B C A B C A A B C A B B
A B C A B C

如图，当我们发现匹配不了时，暴力的做法是从第 6 个位置从头开始，但显然我们不应该这样。我们知道 S 字符串的第一个和第二个不一样，所以也就能知道从 6 开始一定无法匹配。

那我们满足什么条件才有可能匹配呢？我们现在已经匹配了 ABCAB，我们再次匹配时，一定要用到它的一个后缀。也就是说，跳到的地方必须满足从这个地方开始的后缀，等于 S 的前缀：



这个时候，我们只要从第 10 个位置开始看即可。这个位置之前没有看过，不难发现每个位置都只会看一次，所以复杂度为 $\mathcal{O}(|T|)$ 。（假设我们知道应该跳到什么地方）

而且注意这个相同的前后缀必须是最长的，这样可以保证找到第一个可能满足条件的位置，否则可能会漏掉一些答案。比如：

A A B A A B A A A B C

A A B A A A

正确 A A B A A A

错误 A A B A A A ✗

定义一个串 S 的 Border 为 S 的一个非 S 本身的子串 B ，满足 B 既是前缀又是后缀。

设 pre_x 表示 S 的前 x 个字符组成的字符串的最长 Border 长度。这就是所谓的前缀函数。

前缀函数

问题转化为如何快速求前缀函数。

注意到一个性质：

性质

$pre_{x+1} \leq pre_x + 1$ ，也就是相邻的前缀函数最多 $+1$ 。

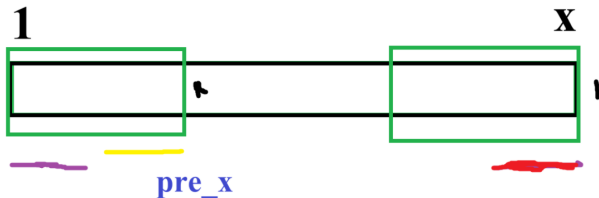
这个性质显然，否则 pre_x 可以更大。

有了这个性质，假设我们已经算出来 $pre_{1...x}$ ，如果 $s_{pre_x+1} = s_{x+1}$ ，我们就可以知道 $pre_{x+1} = pre_x + 1$ 。

考虑如果不相等，一种做法是从长度为 1 开始暴力查看前后缀是否相同。这个复杂度是 $\mathcal{O}(n^2)$ ，可以自己证明，这里掠过。

进一步优化的话，仿照 KMP 的思想，利用好“配对失败”带来的启示。

前缀函数



两个绿色框是相同的，这是 pre 的定义。现在“失配”了，需要重新计算。注意到红色和紫色的部分一定是一样的，然后又由于绿色相同，所以黄色和红色相同。于是紫色和黄色部分一定要相同。注意到这其实是前 pre_x 个字符构成的字符串的相同前后缀，所以长度为 pre_{pre_x} 。同时，保证了紫色和黄色相同也保证了紫色和红色相同，所以只要 $s[pre_{pre_x} + 1] = s[x + 1]$ ，就有 $pre_{x+1} = pre_{pre_x} + 1$ 。

前缀函数

于是求 pre_{x+1} 就这样做：

1. 如果 $s_{x+1} = s_{pre_x+1}$ ，则 $pre_{x+1} = pre_x + 1$
2. 令 $d = pre_x$ ，不停地跳 $d = pre_d$ 直到 $s_{x+1} = s_{d+1}$ ，则 $pre_{x+1} = d + 1$ 。
3. 如果找不到 $s_{x+1} = s_{d+1}$ ， $pre_{x+1} = 0$ 。

注意到 pre 只会进行 n 次 $+1$ ，而每跳一次， pre 就至少会 -1 ，所以不会跳超过 n 次，总的复杂度 $\mathcal{O}(n)$ 。

回到 KMP，KMP 算法的复杂度为 $\mathcal{O}(|S| + |T|)$ 。

似乎前缀数组本身比 KMP 更常用。

例题 D

D - 【模板】KMP

给定字符串 s_1 和 s_2 。要求：

1. 输出 s_2 在 s_1 中所有出现位置的起始下标（从 1 开始），按升序排列；
2. 对 s_2 的每个前缀，输出该前缀的最长 border 长度。

数据规模： $|s_1|, |s_2| \leq 10^6$ 。

例题 D

D - 【模板】KMP

给定字符串 s_1 和 s_2 。要求：

1. 输出 s_2 在 s_1 中所有出现位置的起始下标（从 1 开始），按升序排列；
2. 对 s_2 的每个前缀，输出该前缀的最长 border 长度。

数据规模： $|s_1|, |s_2| \leq 10^6$ 。

模板题

例题 C 再看

C - Compress Words

Amugae 有一个由 n 个单词组成的句子。他想把这个句子压缩成一个单词。Amugae 不喜欢重复，因此当他将两个单词合并成一个单词时，会移除第二个单词中与第一个单词后缀相同的最长前缀。例如，他将"sample" 和"please" 合并成"samplease"。Amugae 会从左到右依次合并他的句子（即，先合并前两个单词，然后将结果与第三个单词合并，依此类推）。请编写程序输出合并过程结束后得到的压缩单词。

数据规模： $n \leq 10^5$ ， $\sum |s_i| \leq 10^6$ 。

例题 C 另解

这道题目是前缀 = 后缀，这是非常明显的 pre 数组暗示。

设现在要合并 S, T , S 在前，我们可以求 $T + \& + S$ 的 pre 数组， pre_{last} 即为可以删除的长度。特殊字符 $\&$ 是为了避免 pre 太长跨到另一个字符串。

这样复杂度不对。注意到 pre_{last} 不会超过 T 的长度，所以每次 S 只需要截取后 $|T|$ 长度的后缀即可。每次只对长度 $\mathcal{O}(|T|)$ 的串求 pre ，总复杂度 $\mathcal{O}(\sum |s_i|)$ 。

目录

① 字符串哈希

② KMP 算法

③ Border 的性质与单串失配树

Border 的性质

在我们之后的讨论中，用 $Bd(S)$ 表示 S 的 Border 集合， $mxBd(S)$ 表示 S 的最长 Border。

性质 1

$$Bd(S) = mxBd(S) + Bd(mxBd(S))$$

这个定理告诉我们，一个串的全部 Border 可以由不断跳 pre 数组获得。

Border 的性质

考虑证明。对于这种“集合 $A = \text{集合 } B$ ”的形式，我们一般证明 $A \subseteq B$ 并且 $B \subseteq A$ 。
证明 $A \subseteq B$ ，可以证明任取 $x \in A$ ，都有 $x \in B$ 。

- $Bd(S) \supseteq (mx Bd(S) + Bd(mx Bd(S)))$ ，即证明 $mx Bd$ 的每个元素都是一个原串的 Border，这我们之前已经证明。
- $Bd(S) \subseteq (mx Bd(S) + Bd(mx Bd(S)))$ 。设存在一个 Border。如果它不是 $mx Bd(S)$ ，说明其长度 $< pre_{len}$ 。设这个 Border 长度为 d ，则 $S_{1 \sim d} = S_{(len-d+1) \sim len}$ ，而由于 $mx Bd$ 的性质，有 $S_{(len-d+1) \sim len} = S_{pre_{len}-d+1 \sim pre_{len}}$ ，说明这是 $mx Bd(S)$ 的 Border。

这个定理也是非常自然的。

单串失配树

我们注意到 $pre_x < x$ ，则如果我们连边 $pre_x \rightarrow x$ ，可以生成一棵树。取 0 为根，则一个节点 x 的父亲就是 pre_x ，从一个点不断跳 pre 可以得到一个前缀所有的 Border（的长度）。

这个东西叫失配树。为什么叫这个名字？可以参考 KMP 的过程，当失配时，下次已经匹配的长度是 pre_x 。于是我们也可以经常看到用 $fail_x$ 表示前缀数组 pre 的。



树有很多性质，我们也太了解它了，所以这个转化之后我们可以做很多东西。

例题 E

E - 【模板】失配树

给定字符串 s ，长度为 N 。对任意 L ，定义前缀 $p_L = s[1..L]$ 。

m 次询问，每次给出 p, q ，求：

$$\max\{b \mid 1 \leq b < \min(p, q), b \text{ 同时是 } p_p \text{ 和 } p_q \text{ 的 border}\}$$

若不存在，输出 0。

$N \leq 10^6$ ， $m \leq 10^5$ 。

例题 E

模板题。利用失配树不断跳 *pre* 可以得到所有 Border，又因父亲的长度小于子节点的长度，可知两个前缀最长的公共 Border 就是它们的 LCA（所代表的前缀）。

一个小细节，通常的 Border 是不包含本身的，所以如果求出 $LCA(u, v) = u/v$ 时还要再跳一下父节点。

例题 F

F - 动物园

给定字符串 s ，长度 L 。对每个 i ($1 \leq i \leq L$)，求前缀 $s[1..i]$ 中长度不超过 $\frac{i}{2}$ 的 Border 的数量。

总长度 $\leq 10^6$ 。

例题 F 题解

最直接的想法，考虑建出失配树，然后不断跳父亲节点直到长度 $\leq \frac{i}{2}$ ，检查深度。

使用倍增优化可以实现 $\mathcal{O}(L \log L)$ 。可以得到非常可观的部分分。

注意到一个性质，设最长不超过 $\frac{i}{2}$ 的 Border 长度为 len_i ，则 $len_{i+1} \leq len_i + 1$ 。证明的话，如果 $len_{i+1} > len_i + 1$ 则 len_i 可以取到 $len_{i+1} - 1 > len_i$ ，更长。

发现这个性质和 pre 数组性质几乎一样，则我们先求出 pre 数组，然后完全类似地求出 len ，只是如果 $len_i > \frac{i}{2}$ 就跳 pre 。 num 就是 Fail 树的深度。总的复杂度 $\mathcal{O}(n)$ 。

Border 性质 2 - 求循环节

给出定义：我们认为 $S_{1 \sim p}$ 为循环节（或者叫周期），当任意 $i \in [1, len - p]$ ，都有 $S_i = S_{i+p}$ 。

注意这里不要求循环节是完整的，比如对于 `abbab`，`abb` 也是一个循环节。
对此，有一个经典结论：

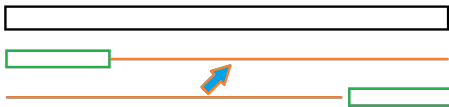
KMP 求循环节

如果 S 有一个长度为 r 的 Border，则它有一个长 $|S| - r$ 的循环节。

Border 性质 - 循环节

考虑证明。其实看这个图即可。箭头表示利用 Border 的性质推出两个圈的部分相同，绿色表示 Border：

情况 1: $r \leq \text{len}/2$



情况 2: $r > \text{len}/2$



这个定理反过来也成立，证明类似。

例题 G

G - Periodic Strings

给定若干个字符串，对每个字符串，求最小的正整数 k ，使得该字符串可由某个长度为 k 的字符串重复拼接而成。

例题 G 题解 - KMP 求循环节

这个题是入门题，暴力即可。

这道题和正常的循环节不同的是，它要求循环节必须完整出现，或者说，循环节长度必须能够整除总长。

一个直接的想法是：不断跳 pre 找出所有的 Border 长度 r ，看有没有 $|S| - r$ 能够整除 $|S|$ 的。

这当然可以，不过没有必要。实际上我们只需要看最长 Border 即可，如果最长 Border 不满足 $|S| - r$ 能够整除 $|S|$ ，则就没有其他 Border 可以。

证明的话，设一个串最短的循环节长度为 x ，它未必可以整除，另一个为 y ，它可以整除，这说明 $y \leq \frac{|S|}{2}$ ，则由一个小性质知道 $\text{GCD}(x, y)$ 也是一个循环节。由于 x 的最小性， $\text{GCD} \geq x$ ，而又由于 $\text{GCD} \leq x$ ，可知 $\text{GCD} = x$ 。

这说明 x 是 y 的因数，说明 x 也可以整除总长度。

周期引理 (Periodicity Lemma)

这个小性质其实是一个有名的引理：

弱周期引理 (Weak Periodicity Lemma)

设 p, q 为 S 的两个循环节（或者叫周期），且满足 $p + q \leq n$ ，则 $\text{GCD}(p, q)$ 也为 S 的循环节。

证明：不妨设 $p < q$ ，则设 $d = q - p$ ，如果证明了 d 是周期，则辗转相减法可以证明原命题，注意到有：

- 对于 $i > p$ ，有 $s_i = s_{i-p} = s_{i-p+q} = s_{i+d}$
- 对于 $i < q$ ，有 $i + p < q + p \leq n$ ，则 $s_i = s_{i+q} = s_{i+q-p} = s_{i+d}$

显然覆盖了所有的 i ，于是证毕。□

这个引理还有一个强的版本，即只要求 $p + q - \text{GCD}(p, q) \leq n$ 。证明复杂，掠过。

*Border 性质

性质 3

一个串 S 长度超过 $\frac{|S|}{2}$ 的 Border 的长度构成等差数列。

证明：设最长 Border 长度为 $n - p$ ，另一个为 $n - q$ ，两个都大于 $\frac{n}{2}$ ，则 $p, q \leq \frac{n}{2}$ ，从而 $p + q \leq n$ 。

由刚才的引理得 $\text{GCD}(p, q)$ 也是周期，且类似 E 题的证明我们知道 $p|q$ (p 整除 q)。于是所有这些 Border 都是关于 p 的同余。而由循环节性质知道 p 的倍数都是周期，所以构成等差数列。□

*Border 性质

性质 4

一个串 S 的所有 Bd 按长度排序后, 可以被划分成 $\mathcal{O}(\log n)$ 个等差数列。

首先, 将该串的长度达到 $n/2$ 的 Bd 划分成一个等差数列。

考虑长度达到 $n/2$ 的 Bd 中长度最小的 T 。

- 若最小循环节 $d \leq n/4$, 不断从最长 Bd 减去 d 则必然有 $|T| \leq \frac{3}{4}n$ 。
- 若 $d > n/4$, 则最长 Bd 本身就 $\leq \frac{3}{4}n$ 。

问题缩小至原来的 $3/4$ 规模。如此缩小, $\mathcal{O}(\log |S|)$ 轮就结束了。

*Border 性质

性质 4

一个串 S 的所有 Bd 按长度排序后, 可以被划分成 $O(\log n)$ 个等差数列。

首先, 将该串的长度达到 $n/2$ 的 Bd 划分成一个等差数列。

考虑长度达到 $n/2$ 的 Bd 中长度最小的 T 。

- 若最小循环节 $d \leq n/4$, 不断从最长 Bd 减去 d 则必然有 $|T| \leq \frac{3}{4}n$ 。
- 若 $d > n/4$, 则最长 Bd 本身就 $\leq \frac{3}{4}n$ 。

问题缩小至原来的 $3/4$ 规模。如此缩小, $O(\log |S|)$ 轮就结束了。

由上面的证明不难发现, 这些等差数列的公差是成倍增长的, 所以又有推论:

- 一个串 S 公差 $\geq d$ 的 Bd 等差数列总大小是 $O(\frac{n}{d})$ 的。

* 例题 H

H - JOJO

维护一个字符串（初始为空），支持两种操作：

1. 在末尾追加 x 个相同字符 c ($x \geq 1$)，且若当前串非空，则当前串末字符 $\neq c$ 。
2. 将字符串恢复为第 x 次操作后的状态。

每次操作后，求所有前缀的 pre 之和。

数据规模：操作数 $n \leq 10^5$ ，单次添加的 $x \leq 10^4$ 。

* 例题 H 题解

我们将连续相同的串缩成一个二元组，如连续 4 个 a 表示为 $(a, 4)$ ，把这个东西当成一个字符。我们认为，两个二元组只有全部相同才能认为是相同字符，如 $(a, 4)$ 和 $(a, 5)$ 为两个不同字符。

这样转化后，我们定义 $fail_x$ 为转化后的 pre 数组， $failen_x$ 表示把这个最长 Border 里面的第二维加起来，例如： $(a, 2)(b, 3)(a, 2)(b, 1)(a, 2)(b, 3)$ ，则 $fail = \{0, 0, 1, 0, 1, 2\}$ ，对应 $failen = \{0, 0, 2, 0, 2, 5\}$ 。

为什么要这样做呢？因为对于一个 Border，其最后可能是一个不完整的段，前面一定是完整的部分。

因此，当我们加入一个二元组 (c, x) 时，不断跳 $fail$ 。如果跳到的这个节点编号为 r ，它的下一个字符为 (c', x') ，且 $c = c'$ ，则对于只新加入 $y \leq x'$ 个 c' 的前缀，有长度为 $failen_r + y$ 的 Border。

于是不断跳 Border，然后求 $\max$ ，这样会形成若干个等差数列，可以容易求解。

* 例题 H 题解

我们跳 Border 只需要跳到 $c' = c$ 即可，这样可以求出新的 *fail*，也可以覆盖所有的 y 。

如果没有回退之类的操作，这样暴力做的复杂度是对的。但是这道题有回退，或者说可持久化，可能会在各种 *fail* 需要跳很多次的地方反复询问，使得复杂度爆炸。利用刚才的性质。Border 的长度可以被划分成 $\mathcal{O}(\log n)$ 个等差序列。而且我们由刚才的证明知道，每串等差序列，中间夹着的部分一定是“循环重复”的。既然是重复的，那也就没必要全部查看，只需要查看最长的，然后直接跳至下一个等差序列即可。

每次跳 *fail* 的次数严格 $\mathcal{O}(\log n)$ ，总跳跃次数就是 $\mathcal{O}(n \log n)$ 的。

离线下来，建出依赖树（按回退/操作历史建树），每次加入/撤销即可，不需要可持久化数据结构。